

university of groningen

Performance Engineering: Lecture #06

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-17

Overview



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Performance Metric

- **Parallel speedup** using P processes

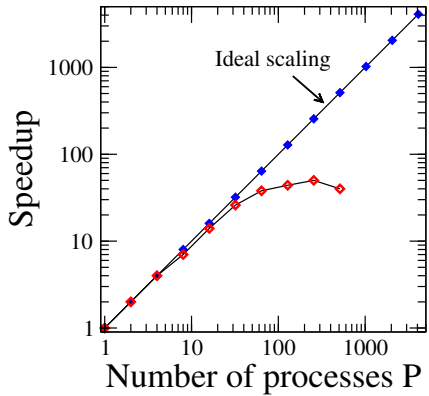
$$S(P) = \frac{\Delta t_{\text{serial}}}{\Delta t_{\text{parallel}}(P)}$$

- **Parallel efficiency**

$$\epsilon = \frac{S(P)}{P}$$

Example: $S = 26$ using $P = 32$ processing elements
$\Rightarrow \epsilon = 81\%$

- **Super-linear speedup**: $S > P$



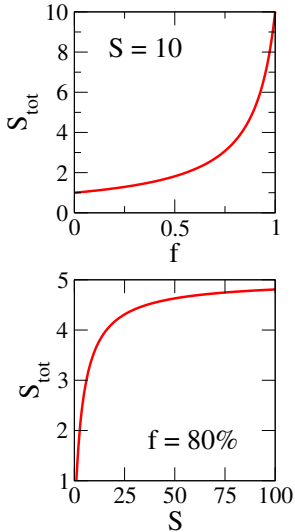
Amdahl's Law

- Consider the following case:
  - Fraction f of application can be accelerated resulting in speed-up by factor S
  - The remaining fraction $1 - f$ remains serial
- Execution time after acceleration:

$$\Delta t' = (1 - f)\Delta t + \frac{f\Delta t}{S}$$

- **Total speed-up**

$$S_{\text{tot}} = \frac{\Delta t}{\Delta t'} = \frac{1}{(1 - f) + f/S}$$



Gustafson's Law



- Amdahl's law: Only considers scaling with fixed workload ☞ **strong scaling**
- Different ansatz by Gustafson:
  - Increase workload proportional to number of processing elements P ☞ **weak scaling**
  - Assume non-parallelized code part to be independent of workload
- Execution times
  - Serial execution
    - Execution time for original problem: Δt
    - Execution time for scaled problem size: $(1-f)\Delta t + fP\Delta t$
  - Parallel execution
    - Execution time for scaled problem size: $(1-f)\Delta t + f\Delta t$
- Speed-up:

$$S = \frac{(1-f)\Delta t + fP\Delta t}{(1-f)\Delta t + f\Delta t} = (1-f) + fP$$

Parallel Efficiency: Amdahl vs. Gustafson



- Parallel efficiency according to Amdahl:

$$\epsilon_{\text{Amdahl}}(P) = \frac{1}{P} \cdot \frac{1}{(1-f) + f/P} = \frac{1}{P(1-f) + f}$$

- Parallel efficiency according to Gustafson:

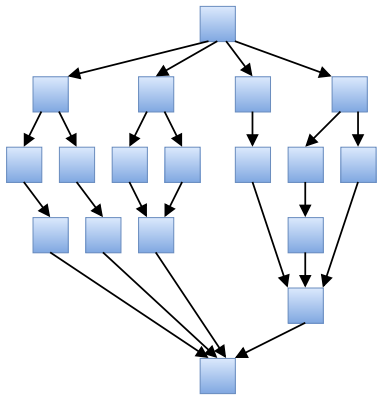
$$\epsilon_{\text{Gustafson}}(P) = \frac{(1-f) + fP}{P} = \frac{1-f}{P} + f$$

- Limit of large number of processing elements P:

$$\begin{aligned} \lim_{P\to\infty} \epsilon_{\text{Amdahl}}(P) &= 0 \\ \lim_{P\to\infty} \epsilon_{\text{Gustafson}}(P) &= f \end{aligned}$$

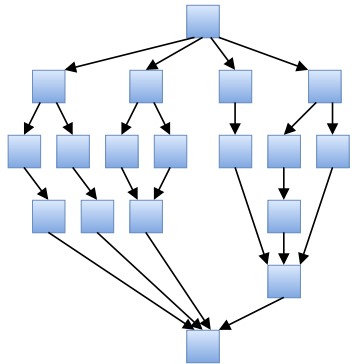
Work-Span Model

- **Consider**: A **task graph**, which is assumed to be a **Directed Acyclic Graph (DAG)**
- **Assumptions** (inspired by the PRAM model)
  - The latency of each task is one time unit
  - Task scheduling is greedy
- **Parallel latency** $\Delta t(P)$ for P processing units
- **Shown example**:
  - Maximum concurrency: 7
  - $\Delta t(P) = 6$ for $P \geq 7$



Work and Span

- $\Delta t(1)$ is called **work of an algorithm**
  - Time needed to executed all task in a serial manner
  - Here: $\Delta t(1) = 18$
- $\Delta t(\infty)$ is called **span of an algorithm**
  - Time needed on an ideal machine with an infinite number of processors
  - Here: $\Delta t(\infty) = 6$
  - Alternative terms: **step complexity**, **depth**



Speed-Up Limit

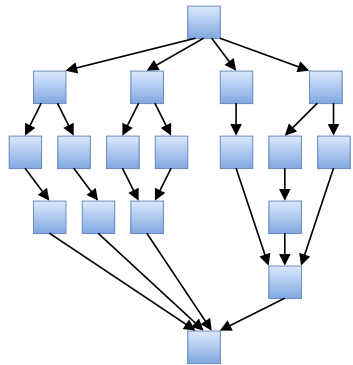
- Super-linear speed-up is not possible in the work-span model

$$S(P) = \frac{\Delta t(1)}{\Delta t(P)} \leq \frac{\Delta t(1)}{\Delta t(1)/P} = P$$

- On an ideal machine with greedy scheduling, adding processors never slows down an algorithm

$$S(P) = \frac{\Delta t(1)}{\Delta t(P)} \leq \frac{\Delta t(1)}{\Delta t(\infty)}$$

  - In words: speed-up $\leq$ work / span



Brent's Lemma



- **Brent's Lemma** provides a lower bound for speed-up [Richart P. Brent; 1964]

$$\Delta t(P) \leq \frac{\Delta t(1) - \Delta t(\infty)}{P} + \Delta t(\infty)$$

- **Interpretation** of Brent's Lemma
  - **Imperfectly parallelizable work**, which always takes $\Delta t(\infty)$
  - **Perfectly parallelizable work**, which for $P = 1$ takes the remaining time $\Delta t(1) - \Delta t(\infty)$
- **Note**: $\Delta t(\infty)$ depends on the concurrency
  - Not to be confused with the serial execution time Δt_{serial} in Amdahl's Law
  - Increasing concurrency will reduce $\Delta t(\infty)$
  - In practice, excess parallelism (**parallel slack**) may result in more efficient task allocation

Comparison: Amdahl versus Brent



- Amdahl's argument results in a lower bound for $\Delta t(P)$
  - Bound applies in case of ideal parallelization
  - Incentive to think about more scaling ("HPC view")
- Brent's Lemma puts an upper bound for $\Delta t(P)$
  - It says what happens if task dependencies are taken into account

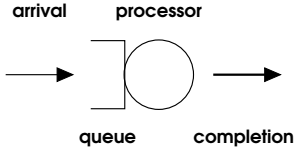
Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Network Queuing Models

- **Machine model**: Network of queues



- **Empirical ansatz**: System is observed for time period T
- **Observables**:

B	Time during which resource was busy
A	Number of observed request arrivals
C	Number of observed request completions
W	Average waiting time

Derived Quantities

- Utilization
$$U = \frac{B}{T}$$
- Throughput = completion rate
$$X = \frac{C}{T}$$
- Average service requirement
$$S = \frac{B}{C}$$
- Average number of requests in the system
$$N = \frac{W}{T}$$
- Average system residence time
$$R = \frac{W}{N}$$

Utilization Law

$$\frac{B}{T} = \frac{C}{T} \cdot \frac{B}{C} \quad \Rightarrow \quad U = XS$$

Utilization Law = The utilization of a resource is equal to the product of the throughput of that resource and the average service requirement of that resource.

Example:

- Disk services 40 requests per second
- Servicing a request requires 0.02 seconds
- Disk utilization is 80 %

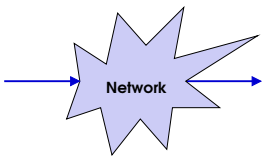
Little's Law

$$\frac{W}{T} = \frac{C}{T} \cdot \frac{W}{C} \quad \Rightarrow \quad N = XR.$$

Little's Law = Average number of requests in a system is equal to the product of the throughput of that system and the average time spent in that system by a request

Example:

N = Average number of packets in network
X = Packet injection rate
R = Average amount of time packet spends in network



Queuing Delay (1/2)



- Estimated average time of staying in the queue (**residence time**):

$$R = N \cdot S + S$$

- Apply Little's Law and solve for R:

$$R = (X \cdot R) \cdot S + S \quad \Rightarrow R = \frac{S}{1 - X \cdot S}$$

- Consider system utilization:

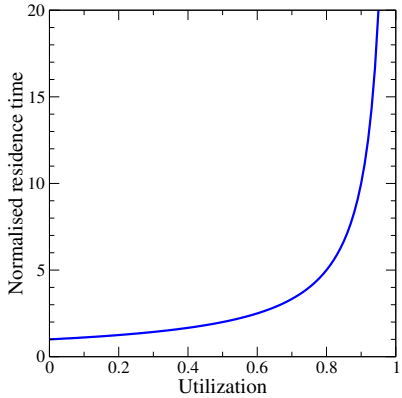
$$U = \frac{B}{T} = \frac{B}{C} \cdot \frac{C}{T} = S \cdot X$$

Queuing Delay (1/2)

- Determine normalised residence time as a function of utilization:

$$\frac{R}{S} = \frac{1}{1-U}$$

☞ **High utilization leads to diverging queuing delays**



Kendall's Notation

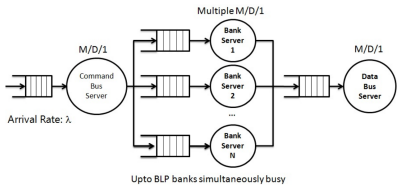


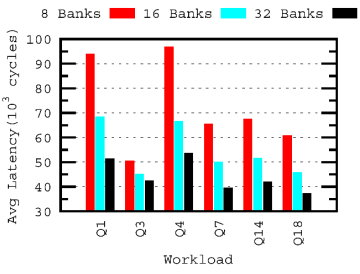
- Queues are characterised by (at least) the following parameters
  - A: Arrival process
    - M: Markovian (Poisson distribution)
    - D: Degenerate distribution
    - ...
  - S: Service time distribution
    - M: Markovian (exponential distribution)
    - D: Degenerate distribution
    - ...
  - c = Number of servers
- Kendall's notation: A/S/c
  - Example: M/D/1 queue
    - Arrivals occur at rate λ according to a Poisson process
    - Service times are deterministic (service rate $\mu = 1/D$)

Queuing Theory Example: DRAM Modelling

- Model ansatz: DRAM channel as a 3-stage network of queues
  - Stage 1: command bus
    - Memory requests arrive following an Poisson distributed (arrival rate $\lambda = A/T$)
  - Stage 2: memory banks
    - Set of bank servers to model the actual servicing of any request by the memory
  - Stage 3: data bus
    - Data bus server which captures the actual data transfer on the channel

[Gulur et al., 2014]





Content



1. Parallel Performance
2. Queueing Theory
3. **Simulation Models**
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Simulation Models



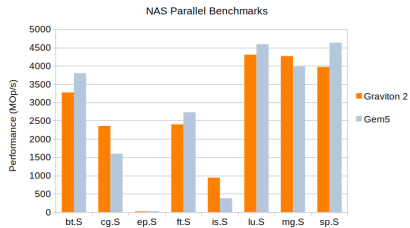
- Simulation models require a simulator for creating observables
- **Examples**:
  - **Cache simulators**: Simulation of cache behaviour
  - Full CPU **discrete event simulators**, e.g. gem5
  - **Hardware Description Language (HDL) simulators**: Signal-level simulations

Event Simulator: GEM5

[Binkert et al.]

- Modular simulator for different CPU models and attached memory
- Support for different CPU models using different ISA: Alpha, ARM, SPARC, x86
- Support of different scopes
  - System-call Emulation (SE)
  - Full-System (FS):
    model complete system including OS
- Different memory models

[Brank, Pleiter; 2023]



Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. **Relevant Hardware Architecture Concepts**
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Parallel Computer Architectures



- **Microarchitecture level**
  - Pipelines
  - Instruction-level parallelism
  - Data-level parallelism: SIMD/vector instructions
  - Simultaneous multithreading (SMT)
- **Chip level**
  - Multi-core architectures
  - Memory-level parallelism
- **Node level**
  - Multi-socket
- **System level**
  - Multi-node

Pipeline: Time Diagram

	P_0	P_1	P_2	P_3	P_4
t_0	C_0				
t_0+1	C_1	C_0			
t_0+2	C_2	C_1	C_0		
t_0+3	C_3	C_2	C_1	C_0	
t_0+4	C_4	C_3	C_2	C_1	C_0
t_0+5	C_5	C_4	C_3	C_2	C_1
t_0+6		C_5	C_4	C_3	C_2
t_0+7			C_5	C_4	C_3
t_0+8				C_5	C_4
t_0+9					C_5

- **pipeline filling**:
  $t = t_0 ... (t_0 + 3)$
- **pipeline draining**:
  $t = (t_0 + 6) ... (t_0 + 9)$
- Time to perform n operations in k **pipeline stages** assuming a nominal throughput $B = 1$:

$$\Delta t_p(n, k) = n + (k - 1)$$

Here: $\Delta t_p(6, 5) = 10$

Pipeline Costs



- In our example: throughput $B = 1$
- **Throughput**:

$$b(n,k) = \frac{\text{number of operations}}{\Delta t_p(n,k)} = \frac{n}{n+(k-1)}$$

  Note: $\lim_{n\to\infty} b(n,k) = B$
- **Gain (or speed-up)**

$$s(n,k) = \frac{\text{scalar execution time}}{\text{pipelined execution time}} = \frac{n\,k}{n+(k-1)}$$

  Note: $\lim_{n\to\infty} s(n,k) = k$
- **Efficiency**

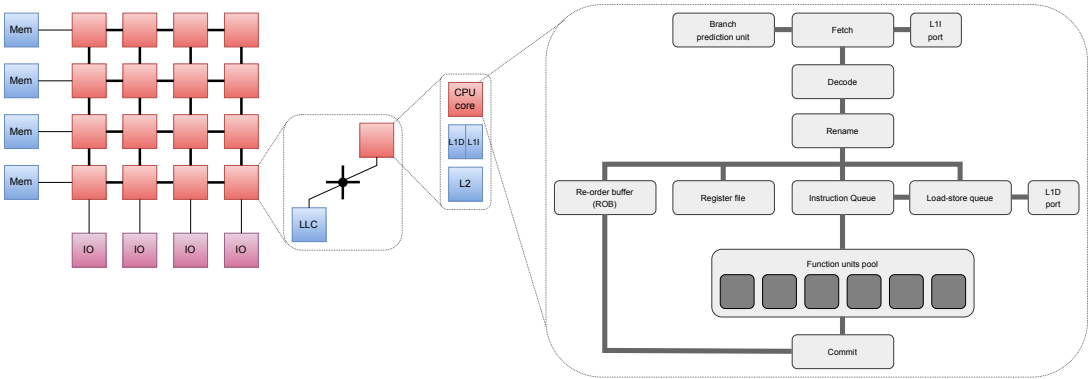
$$\epsilon(n,k) = \frac{1}{k}s(n,k) = \frac{n}{n+(k-1)}$$

Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. **Processor Microarchitecture: Overview**
6. Processor Microarchitecture: Front-end
7. Final

Example: Many-Core CPU Architecture



Instruction Pipeline Stages



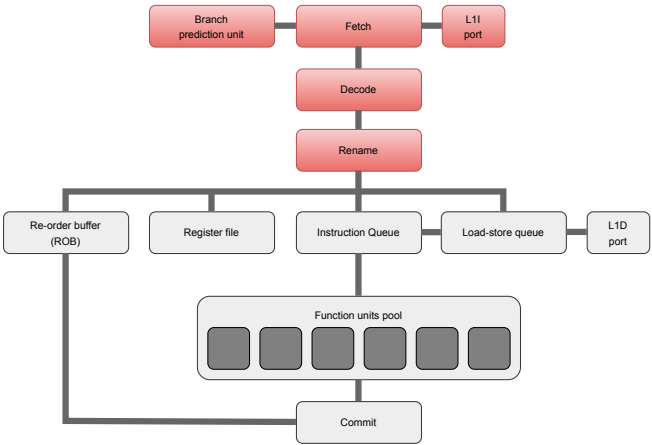
- **Fetch**: Retrieve the instruction from memory/instruction cache
- **Decode**: Interpret the instruction
- **Issue/Dispatch** (optional): Forward the front entry of the instruction pipeline to a free reservation station
- **Execute**: Execute the instruction
- **Write**: Write the results
- **Commit/Retire** (optional): Make results visible

Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. **Processor Microarchitecture: Front-end**
7. Final

Microarchitecture Overview: Front-End



Instruction Fetch and Decode



- **Fetching** typically means reading a cache line from L1I cache and pushing the instructions into a pipeline
  - The throughput per cycle is limited
  - Branches may result in long latencies due to
    - Refilling the pipeline
    - Instruction cache misses
- Instruction **decoding** is usually parallelised
  - Decoding of complex instructions may take longer
  - Instructions may be decoded to multiple **micro-instructions** (also known as **micro-ops**)
- **Performance engineering potential**: Low

Branching and Speculative Execution



- **Branch instructions** cause interrupts in sequential execution of instructions
  - Branches may be **unconditional** or **conditional**
- Mitigation strategy: **branch prediction** in combination with **speculative execution**
  - **Branch prediction** = Mechanisms by which processors predict the outcome of a conditional branch before the actual condition is fully evaluated
  - **Speculative execution** = Facilitate execution of an instruction based on branch prediction by ensuring that commit only happens once condtion is fully evaluated
- **Performance engineering potential**: High

Branch Prediction



- Branch prediction mechanisms
  - Branch hint instructions (software)
  - Hardware that allows processor to learn from past behaviour
    - Information stored in Branch History Tables (BHT)
    - Today, multi-level predictors are used
- Hardware branch predictors will struggle in the following situations:
  - Highly irregular branches
  - First-time branches
  - Indirect branches: branch target is not fixed
  - Difficult to detect data dependencies
- **Performance engineering potential**: Low

Register Renaming



- **Architected registers** = Set of registers foreseen by the ISA
- **Physical registers** = Set of registers implemented in hardware

$$N_{\text{architected registers}} \leq N_{\text{physical registers}}$$

- Hardware may support renaming of architected registers such that all physical registers are used
- **Performance engineering potential**: Low

Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. **Final**

Summary Lecture #06



- Modelling techniques
  - Parallel performance: Amdahl's Law, Gustafson's Law, work-span model
  - Queuing theory models
  - Simulation models
- Reviewed key architectural concepts like pipelining, parallelisation
- Review of key elements of processor microarchitectures with the following having the highest performance engineering potential
  - Front-end: Branching

Questions?



[Wiki Media]